[WIP] Intrernal Tools UI Standardization

Project Background

The primary goal of this UI Standardization project is to transition from isolated, inconsistent internal applications to a unified **Design System** that provides a single source of truth for both designers and developers.

- **The Problem**: Internal tools are often developed by different teams at different times, resulting in "styling drift" where buttons, menus, and data tables look and behave differently across tools. This forces employees to "re-learn" how to use every new tool, increasing cognitive load and the likelihood of errors.
- **The Opportunity**: By implementing standardized components (like buttons, navigation bars, and forms), you can build a library of reusable assets. This allows developers to focus on the **business logic** rather than reinventing the interface for every new feature.
- **The Impact**:
  - **Reduced Development Costs**: Developing once and reusing components significantly cuts down on front-end coding hours.
  - **Higher Efficiency**: A consistent UI allows users to perform tasks faster because they can rely on established patterns and habits.
  - **Easier Maintenance**: Updates to a standard component (e.g., changing a brand color or fixing a bug) automatically propagate to all tools using that library.

Key Objectives

1. **Conduct a Visual Audit**: Catalog existing tools to identify common patterns and the most frequent UI discrepancies.
2. **Define Visual Guidelines**: Establish a set of core colors, typography, and spacing rules that reflect your brand and ensure accessibility.
3. **Build a Reusable Component Library**: Develop a set of "building blocks" (buttons, cards, menus) that can be easily integrated into any new internal project.
4. **Create Documentation**: Use tools like Storybook to provide live, interactive documentation so other developers can see how to use the standard components.

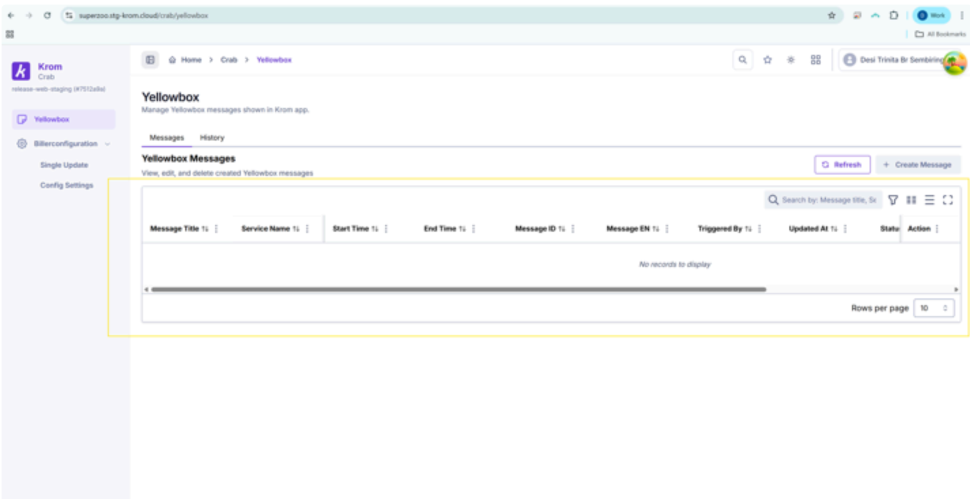
Components

1. **Table**

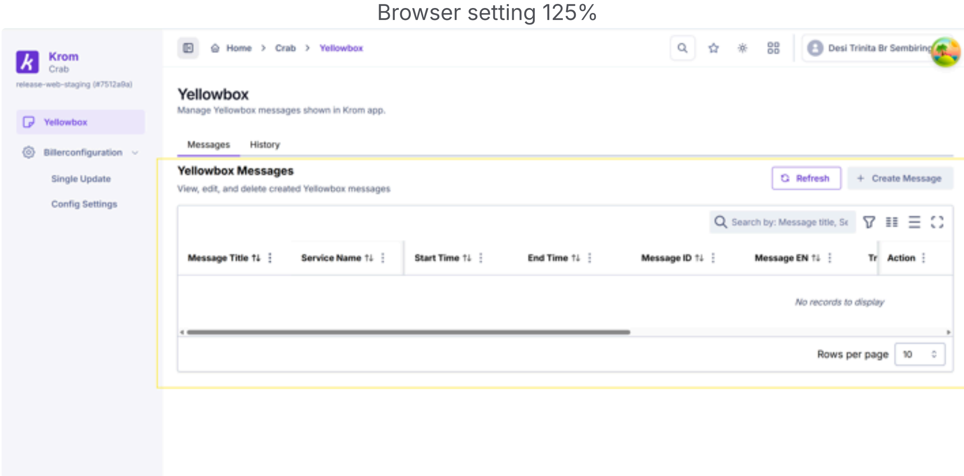
a. **Width**

To maximize data visibility, tables should span the **full width** of the container with **tight padding**. The layout must be **fully responsive**, automatically adjusting to fit the user's screen size and browser settings.

Browser setting 100%



Browser setting 125%



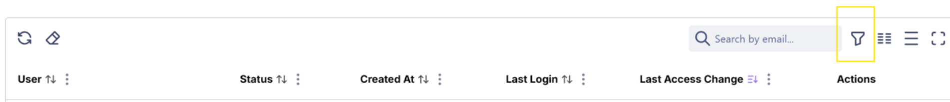
b. **Column Header Sort**

To enhance data discoverability, implement **sortable column headers** for all key data points. While columns containing action buttons, icons, or long-form descriptions should remain static to ensure a clean interface, the final selection of sortable attributes will be determined by **Product Manager (PM)** requirements to ensure the tool aligns with specific business workflows

c. **Column Filter**

Tables should include a **Filter Function** that is **hidden by default** to keep the interface clean. Users can activate the filter controls visibility as needed to perform specific searches or data refinements. the final selection of filterable attributes will be determined by **Product Manager (PM)** requirements to ensure the tool aligns with specific business workflows

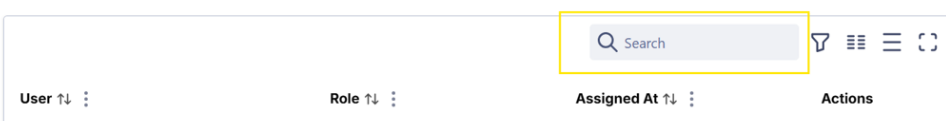
Default



Active Filter

d. **Search Function**

Include a **Search Bar** above the table if the dataset exceeds 50 rows or contains unique identifiers (like IDs,emails, username). The search must execute a **multi-column scan** to return matches instantly as the user types. For dataset that will only show less than 50 rows search bar become optional



e. **Freeze**

For wide tables (after max width still need horizontal scroll to see the data), the primary identifier column should be **frozen on the left** (max 2 columns) so users can track which row they are viewing while scrolling horizontally also The 'Actions' column (if any) should be **frozen on the right** (max 2 columns) of wide tables to ensure critical operations are always accessible without horizontal scrolling

f. **Status**

For tables with a status column, use colored chips to clearly differentiate operational states.

- **purple** (#6936D3) for positive outcomes (*Active, Approved, Done*),

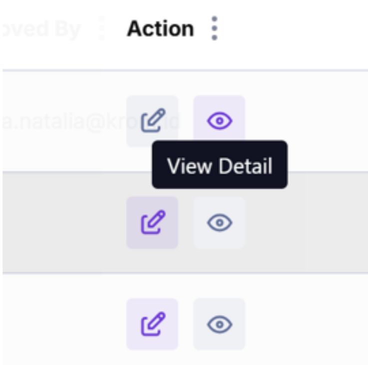
- **blue (**#64B5F6) for intermediate states (*Pending, Needs Approval*),

- **gray (**#808080) for negative or inactive conditions (*Inactive, Rejected, Failed*).
- **red (**#DC2626) to ensure immediate visual priority or any status indicating that **attention is needed**

Hex	Sample	Usage
purple (#6936D3)	BADGE	for positive outcomes (*Active, Approved, Done*),
blue (#64B5F6)	BADGE	for intermediate states (*Pending, Needs Approval*),
gray (#808080)	BADGE	for negative or inactive conditions (*Inactive, Rejected, Failed*).
red (#DC2626)	BADGE	to ensure immediate visual priority or any status indicating that **attention is needed**

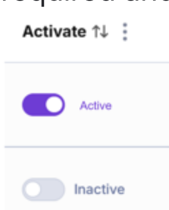
g. **Single Action**

For tables featuring an action column (*View, Edit, Delete, Approve, download, etc*), utilize individual interactive icons (Light Type) paired with descriptive **tooltips** to maintain a clean layout. Apply **purple** (#6936D3) to indicate enabled actions, and switch the icon color to **gray (**#808080) when an action is disabled or unavailable to the user.



h. **Enable/Disable Action**

To streamline operations, tables with binary status controls should feature an inline **Toggle component** to enable or disable features instantly. This reduces the number of clicks required and provides immediate visual feedback on the system's operational state



i. **Global/Bulk Action**

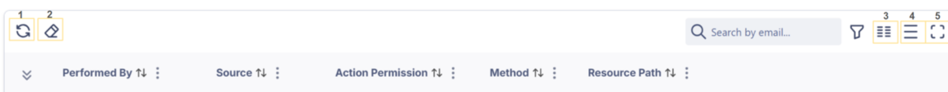
Global configurations or bulk operations, position the primary icon buttons in the **upper-right area** directly above the table layout. These controls should utilize the standard **purple theme** to indicate they are active, functional global actions

j. **Pagination**

Implement server-side pagination for all tables containing more than 10 total records to optimize loading performance and dashboard scannability.

k. **Optional Function**

Following action icons are optional, engineering and PM can decide whether to use this function based on user needs and data complexity



No	Icon Name	Guideline
1	Refresh Data	Include a manual refresh mechanism for real-time or fast-changing data environments to allow users to fetch updates without reloading the entire page
2	Clear Filter	Provide a global reset button when multiple constraints are applied to ensure users can return to the baseline dataset instantly
3	Show/Hide Columns	Enable column customization for wide tables to allow users to control layout density and focus on relevant data
4	Toggle Density	Provide display density controls for data-heavy views to accommodate different scanning preferences
5	Toggle Full Screen	Implement a full-screen toggle for complex dashboards to give users an immersive workspace when analyzing extensive tables